# Práctica 2: Reconocimiento Activo

**Aarón Peñafiel Pacheco Universidad Europea de Madrid Abril 2026**

## Resumen

Esta práctica tiene como objetivo aplicar técnicas de reconocimiento activo sobre entornos simulados con Docker. Se divide en dos partes: primero, la implementación de una función en Python con Scapy para descubrir hosts activos en una red mediante tres protocolos diferentes; y segundo, el análisis del comportamiento por defecto de nmap para el descubrimiento de puertos y su estado.

A diferencia del reconocimiento pasivo, estas técnicas generan tráfico directo hacia los objetivos, por lo que se realizan exclusivamente sobre entornos controlados.

# Contents

Práctica 2: Reconocimiento Activo . . . 1
Introducción . . . 4
Desarrollo . . . 5
Parte 1: Descubrimiento de hosts con Scapy . . . 5
Infraestructura . . . 5
Función `craft_discovery_pkts` . . . 5
Ejemplo de uso . . . 6
Parte 2: Comportamiento por defecto de nmap . . . 7
Infraestructura . . . 7
Estado de un puerto . . . 7
Comportamiento por defecto de nmap . . . 8
Resultados . . . 9
Parte 1: Descubrimiento de hosts . . . 9
Host activo . . . 9
Host inactivo . . . 9
Parte 2: nmap . . . 10
Resultados del escaneo . . . 10
Puerto abierto (80/tcp) . . . 10
Puerto cerrado (443/tcp) . . . 10
Puerto filtrado . . . 11
Evidencia del tráfico completo . . . 11
Conclusiones . . . 12
Bibliography . . . 13

# Introducción

El reconocimiento activo es una fase fundamental en cualquier auditoría de seguridad. Consiste en interactuar directamente con los sistemas objetivo para obtener información sobre su estado, servicios expuestos y configuración de red.

En esta práctica se trabaja sobre dos herramientas complementarias: *Scapy*, una librería Python que permite construir y enviar paquetes de red a medida, y *nmap*, la herramienta de escaneo de redes más utilizada en el sector.

El entorno de pruebas consiste en tres contenedores Docker que exponen servicios reales: un servidor web nginx (HTTP), un servidor FTP y un servidor SSH.

# Desarrollo

## Parte 1: Descubrimiento de hosts con Scapy

### Infraestructura

Para esta parte se utiliza la red local de la máquina virtual con IP

`192.168.122.1`

como host activo y

`192.168.122.200`

como IP inactiva para contrastar resultados.

### Función

**`craft_discovery_pkts`**

La función recibe los siguientes argumentos:

- **`protocols`**

  Lista o string con los protocolos a usar (

  `"UDP"`

  ,

  `"TCP"`

  ,

  `"ICMP"`

  ). Obligatorio.

- **`ips`**

  IP o rango destino en formato string. Obligatorio.

- **`pkt_count`**

  Diccionario opcional con el número de paquetes por protocolo. Por defecto 1.

`port`

Puerto para TCP y UDP. Por defecto 80.

Los tres protocolos utilizados funcionan de la siguiente manera:

**UDP** Se envía un paquete al puerto destino. Si el puerto está cerrado el host responde con ICMP "Port Unreachable", confirmando que está activo.

**TCP ACK** Se envía un paquete ACK sin conexión previa. El host responde con RST confirmando que está activo.

**ICMP Timestamp** Se pregunta al host su hora actual. Útil cuando los firewalls bloquean el ping normal (ICMP Echo).

```
from scapy.all import *

def craft_discovery_pkts(protocols, ips, pkt_count=None, port=80):
    packages = [] # Lista de paquetes a enviar
    for protocol in protocols:
        # Si no se pasa pkt_count se construye 1 paquete por protocolo
        if pkt_count == None:
            n = 1
        else:
            n = pkt_count[protocol]

        if protocol == "UDP":
            for i in range(n):
                package = IP(dst=ips) / UDP(dport=port)
                packages.append(package)

        elif protocol == "TCP":
            for i in range(n):
                # flags="A" indica ACK
                package = IP(dst=ips) / TCP(dport=port, flags="A")
                packages.append(package)

        elif protocol == "ICMP":
            for i in range(n):
                # type=13 es ICMP Timestamp Request
                package = IP(dst=ips) / ICMP(type=13)
                packages.append(package)

    return packages
```

### Ejemplo de uso

```
# Host activo
packages = craft_discovery_pkts(["UDP", "TCP", "ICMP"], "192.168.122.1")
answered, unanswered = sr(packages, timeout=2, verbose=0)

for sent, received in answered:
    if sent.haslayer(UDP):
        proto = "UDP"
    elif sent.haslayer(TCP):
```

```
        proto = "TCP"
    elif sent.haslayer(ICMP):
        proto = "ICMP"
    print(f"{received[IP].src} -> {proto} ha respondido")
```

## Parte 2: Comportamiento por defecto de nmap

### Infraestructura

Para esta parte se levantan tres contenedores Docker con la siguiente configuración:

| Contenedor | Servicio | IP |
|---|---|---|
| http-container | nginx (HTTP) | 172.19.0.4 |
| ftp-container | FTP | 172.19.0.3 |
| ssh-container | SSH | 172.19.0.2 |

```
┌──(kalamur1㉿kali)-[~/Laboratorio-hacking-uem/P2_Reconocimiento_activo]
└─$ cat docker-compose.yml
services: # Los contenedores que creamos
  http:
    image: nginx # Servidor web IP: 172.19.0.2
    container_name: http-container
    networks:
      - mi-red

  ssh:
    image: linuxserver/openssh-server # IP: 172.19.0.4
    container_name: ssh-container
    environment:
      - USER_NAME=ssh-user
      - LISTEN_PORT=22
      - PASWORD_ACCESS=true
      - USER_PASWORD=123
    ports:
      - "2222:22"
    networks:
      - mi-red

  ftp:
    image: garethflowers/ftp-server # IP: 172.19.0.3
    container_name: ftp-container
    ports:
      - "21:21"
      - "20:20"
      - "40000-40009:40000-40009"
    environment:
      - FTP_PASS=root
      - FTP_USER=user
    networks:
      - mi-red

networks: # Parte en la que configurams la red
  mi-red: # nombre de la red que compartiran todos los dockers
    driver: bridge # el tipo de red, esté es el defecto comunican con la red y el host
```

### Estado de un puerto

Un puerto puede encontrarse en tres estados posibles:

**Abierto** Hay un servicio escuchando. El host responde con SYN-ACK al SYN de nmap.
**Cerrado** No hay servicio. El host responde con RST-ACK al SYN de nmap.
**Filtrado** Un firewall bloquea el tráfico. No hay respuesta alguna.

| Estado | Estímulo | Respuesta |
|---|---|---|
| Abierto | SYN | SYN-ACK |
| Cerrado | SYN | RST-ACK |
| Filtrado | SYN | Sin respuesta |

**Comportamiento por defecto de nmap**

Cuando se ejecuta nmap sin opciones adicionales realiza un **SYN scan** (también llamado half-open scan o stealth scan) sobre los **1000 puertos más comunes**. Este tipo de escaneo:

- Envía un paquete TCP SYN a cada puerto
- Si recibe SYN-ACK responde con RST para no completar el handshake
- Nunca establece una conexión completa, lo que lo hace más sigiloso

Los puertos se escanean en **orden aleatorio** para dificultar su detección por sistemas IDS/IPS.

# Resultados

## Parte 1: Descubrimiento de hosts

### Host activo

La ejecución del script contra

```
192.168.122.1
```

devuelve respuesta para los tres protocolos:

```
┌──(kalamur1㉿kali)-[~/Practica2.1]
└─$ sudo python3 Práctica2_1.py
192.168.122.1 → UDP ha respondido
192.168.122.1 → TCP ha respondido
192.168.122.1 → ICMP ha respondido

┌──(kalamur1㉿kali)-[~/Practica2.1]
└─$ sudo python3 Práctica2_1.py
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
```

La captura de Wireshark confirma el tráfico generado por los tres protocolos:

| | | | | |
|---|---|---|---|---|
| 37 | 51.735956715 | 192.168.122.145 | 192.168.122.1 | UDP |
| 38 | 51.736239395 | 192.168.122.1 | 192.168.122.145 | ICMP |
| 39 | 51.740572996 | 192.168.122.145 | 192.168.122.1 | TCP |
| 40 | 51.740861050 | 192.168.122.1 | 192.168.122.145 | TCP |
| 41 | 51.743709740 | 192.168.122.145 | 192.168.122.1 | ICMP |
| 42 | 51.743987560 | 192.168.122.1 | 192.168.122.145 | ICMP |

### Host inactivo

Al ejecutar contra

```
192.168.122.200
```

no se obtiene ninguna respuesta. Scapy muestra warnings de MAC no encontrada ya que el host no existe en la red.

## Parte 2: nmap

### Resultados del escaneo

```
┌──(kalamur1㉿kali)-[~/Laboratorio-hacking-uem/P2_Reconocimiento_activo]
└─$ sudo nmap 172.19.0.4
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-20 13:16 +0200
Nmap scan report for 172.19.0.4
Host is up (0.000013s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http
MAC Address: D6:16:DF:30:5E:6F (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.74 seconds

┌──(kalamur1㉿kali)-[~/Laboratorio-hacking-uem/P2_Reconocimiento_activo]
└─$ sudo nmap 172.19.0.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-20 13:24 +0200
Stats: 0:00:00 elapsed; 0 hosts completed (0 up), 1 undergoing ARP Ping Scan
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 172.19.0.3
Host is up (0.000014s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
MAC Address: DA:82:BE:A1:46:6D (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.77 seconds

┌──(kalamur1㉿kali)-[~/Laboratorio-hacking-uem/P2_Reconocimiento_activo]
└─$ sudo nmap 172.19.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-20 13:24 +0200
Nmap scan report for 172.19.0.2
Host is up (0.000013s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 9A:45:BB:AB:C9:BD (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.72 seconds
```

| Host | Puerto | Servicio |
|---|---|---|
| 172.19.0.4 | 80/tcp | HTTP |
| 172.19.0.3 | 21/tcp | FTP |
| 172.19.0.2 | 22/tcp | SSH |

### Puerto abierto (80/tcp)

La captura de Wireshark muestra el intercambio SYN → SYN-ACK → RST característico del SYN scan:

| | | | | | |
|---|---|---|---|---|---|
| 49 | 4.613244794 | 172.19.0.1 | 172.19.0.4 | TCP | 58 49510 → 80 [SYN] Seq=0 Win=1024 Len=0 |
| 50 | 4.613294397 | 172.19.0.4 | 172.19.0.1 | TCP | 58 80 → 49510 [SYN, ACK] Seq=0 Ack=1 Win |
| 51 | 4.613302851 | 172.19.0.1 | 172.19.0.4 | TCP | 54 49510 → 80 [RST] Seq=1 Win=0 Len=0 |

### Puerto cerrado (443/tcp)

Al escanear el puerto 443 de nginx, el host responde con RST-ACK confirmando que está cerrado:

| | | | | | | |
|---|---|---|---|---|---|---|
| 7 | 4.612331861 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 443 [SYN] Seq=0 Win=1024 |
| 8 | 4.612341518 | 172.19.0.4 | 172.19.0.1 | TCP | 54 | 443 → 49510 [RST, ACK] Seq=1 Ack= |

### Puerto filtrado

Al escanear una IP inexistente (

```
172.19.0.100
```

), nmap solo genera tráfico ARP sin respuesta. No llega a escanear puertos porque no puede confirmar que el host esté activo:

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000000 | 92:53:29:d4:ee:b8 | Broadcast | ARP | 42 | Who has 172.19.0.10 |
| 2 | 1.201644888 | 92:53:29:d4:ee:b8 | Broadcast | ARP | 42 | Who has 172.19.0.10 |
| 3 | 98.704051471 | 92:53:29:d4:ee:b8 | Broadcast | ARP | 42 | Who has 172.19.0.10 |
| 4 | 99.905558843 | 92:53:29:d4:ee:b8 | Broadcast | ARP | 42 | Who has 172.19.0.10 |

### Evidencia del tráfico completo

| | | | | | | |
|---|---|---|---|---|---|---|
| 1942 | 4.641769139 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 7741 [SYN] Se |
| 1944 | 4.642269843 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 14442 [SYN] S |
| 1946 | 4.642338131 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 1864 [SYN] Se |
| 1948 | 4.642373444 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 119 [SYN] Seq= |
| 1950 | 4.642407044 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 7070 [SYN] Se |
| 1952 | 4.642437813 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 49160 [SYN] S |
| 1954 | 4.642469068 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 9998 [SYN] Se |
| 1956 | 4.642541696 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 82 [SYN] Seq= |
| 1958 | 4.642577158 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 49167 [SYN] S |
| 1960 | 4.642614336 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 777 [SYN] Seq= |
| 1962 | 4.642653060 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 5633 [SYN] Se |
| 1964 | 4.642688994 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 5960 [SYN] Se |
| 1966 | 4.642721940 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 9200 [SYN] Se |
| 1968 | 4.642753010 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 4662 [SYN] Se |
| 1970 | 4.642784262 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 48080 [SYN] S |
| 1972 | 4.642815819 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 1782 [SYN] Se |
| 1974 | 4.642848456 | 172.19.0.1 | 172.19.0.4 | TCP | 58 | 49510 → 8181 [SYN] Se |

## Conclusiones

Esta práctica ha demostrado el funcionamiento de dos aproximaciones complementarias al reconocimiento activo.

En la parte 1, la función

```
craft_discovery_pkts
```

permite construir paquetes personalizados con tres protocolos distintos para detectar hosts activos. Cada protocolo tiene ventajas diferentes — TCP ACK es más sigiloso, ICMP Timestamp puede evadir firewalls que bloquean el ping normal, y UDP permite detectar hosts que no responden a TCP.

En la parte 2, se ha comprobado que nmap realiza por defecto un SYN scan sobre los 1000 puertos más comunes, en orden aleatorio y sin completar el handshake TCP. Las respuestas permiten clasificar cada puerto como abierto, cerrado o filtrado según el tipo de paquete recibido.

# Bibliography